TABLERO DE TAREAS KANBAN

Informe Técnico Final de Verificación y Validación de Software

Materia:	Verificación y Validación de Software
Curso:	3° Año — Tecnicatura Superior en Desarrollo de Software (TSDS)
Institución:	Instituto de Educación Superior
Fecha de entrega:	01/11/2026
Proyecto / Repo:	proyecto final_aimar (FastAPI + SQLite / React + Vite)
Estado Final:	100% de Casos Aprobados (105/105) — 0 Defectos Abiertos

RESUMEN EJECUTIVO DE ENTREGA

Este documento constituye el informe técnico final integral que consolida y documenta formalmente todo el proceso de Análisis, Diseño, Desarrollo, Verificación, Validación, Automatización, TDD, Refactorización, Regresión y Seguimiento de Defectos para el sistema Gestor de Tareas Kanban.

2. Integrantes del Equipo

El proyecto fue desarrollado de forma colaborativa por el equipo integrado por los siguientes alumnos:

Integrante	Rol Principal y Responsabilidades
Aureliano Eichhorn	Diseño de base de datos (SQLite), API REST en FastAPI, esquemas Pydantic, reglas de negocio en repository, inicialización y migraciones aditivas, y scripts de pruebas de API con httpx.
Nicolás Delfino	Desarrollo de interfaz gráfica en React 19 + Vite, componentes de tablero Kanban, drag and drop, modales, formularios, hooks de comunicación HTTP y pruebas E2E con Playwright.

Nota: Ambos integrantes realizaron revisiones cruzadas de código (peer code reviews) e inspecciones cruzadas de artefactos antes de cada entrega.

3. Descripción del Problema

El equipo de trabajo detectó la necesidad de contar con una herramienta ligera, ágil e intuitiva para **organizar, gestionar y hacer seguimiento visual** del estado de distintas actividades diarias mediante un tablero tipo Kanban.

Usuarios objetivo: Estudiantes, profesionales independientes y pequeños equipos de trabajo que requieren gestionar sus flujos de tareas sin la complejidad sobrecargada de plataformas corporativas pesadas.

Necesidades clave identificadas: Registrar y editar tareas, controlar avance con subtareas (checklist), adjuntar archivos y enlaces, clasificar por prioridad, responsable y etiquetas, consultar historial de cambios, y realizar búsquedas, filtros y ordenamientos en tiempo real.

4. Objetivos del Proyecto

Objetivo General: Desarrollar y validar un sistema gestor de tareas tipo Kanban visual con backend en FastAPI y frontend en React, aplicando una rigurosa estrategia de Verificación y Validación (V&V;) que garantice la calidad y el cumplimiento de requisitos.

Objetivos Específicos:

- Implementar integralmente los 23 requisitos funcionales (RF1 a RF23) definidos en la especificación.
- Diseñar y ejecutar baterías de pruebas estáticas, dinámicas, unitarias, de integración, de caja negra, de caja blanca y de sistema.
- Construir un arnés de automatización reproducible con pytest, httpx y Playwright que supere los 100 casos automatizados.
- Aplicar prácticas formales de TDD, refactorización y pruebas de regresión sin romper contratos públicos.
- Identificar, registrar y corregir el 100% de los defectos detectados en código y documentación.

5. Requisitos Funcionales y Restricciones

El sistema fue especificado en la Etapa 1 y ampliado en la Etapa 2 con 23 Requisitos Funcionales (RF1 a RF23) y un conjunto de restricciones estrictas de integridad:

Grupo de RFs	Descripción y Reglas de Negocio
RF1 - RF3	Alta de tareas, estado inicial 'Pendiente', actualización de estado (Pendiente, En curso, Terminada).
RF4 - RF5	Reordenamiento de posiciones dentro de la columna (sin huecos) y edición de título, descripción y fecha límite.
RF6 - RF9	Checklist de subtareas (alta, marcado, borrado) y bloqueo de finalización (HTTP 409) si hay subtareas pendientes.
RF10 - RF13	Fecha límite y estado overdue calculado; gestión de adjuntos (subida, descarga, borrado; máx 3 archivos de 5MB).
RF14 - RF15	Eliminación de tareas con borrado lógico (deleted_at) y función de restauración ('Deshacer').
RF16 - RF19	Clasificación enriquecida: Prioridad (Baja, Media, Alta), Responsable (máx 60 char), Etiquetas (máx 5) y Enlaces (máx 5).
RF20 - RF23	Historial de actividad (últimas 20 entradas), Búsqueda/Filtros/Orden, y Alta Enriquecida en un solo POST.

6. Descripción del Sistema y Arquitectura

El sistema es una aplicación web distribuida en dos módulos independientes comunicados mediante la API REST:

- **Backend (API REST):** Desarrollado en Python 3.12 con FastAPI. Sigue una arquitectura en capas estrictamente unidireccional: *Routers (HTTP)* → *Schemas (Pydantic)* → *Repository (Reglas)* → *Database (SQLite)*.
- **Frontend (Interfaz Web):** Desarrollado en React 19 + Vite 8. Presenta el tablero Kanban con tres columnas fijas (Pendiente, En curso, Terminada), modales interactivos y comunicación vía Fetch API con proxy de desarrollo.
- **Modelo de Datos (SQLite):** Conformado por 7 tablas (tasks, subtasks, attachments, tags, task_tags, links, activity) con claves foráneas en cascada y restricciones CHECK.

7. Tecnologías Utilizadas

Capa / Ámbito	Tecnología y Versión	Uso Principal
Capa Frontend	React 19, JavaScript (ES6+), CSS3 custom, Vite 8	Interfaz responsiva del tablero Kanban.
Capa Backend	Python 3.12, FastAPI 0.136.3, Pydantic 2.13.4	API REST, esquemas DTO y validaciones.
Base de Datos	SQLite 3.45.1, Uvicorn 0.49.0 (ASGI)	Persistencia relacional y ejecutor ASGI.
Pruebas & V&V;	pytest 9.2.0, httpx 0.28.1, Playwright 1.64.1	Suites unitarias, integración, API y UI.
Análisis Estático	Ruff 0.17.3, ESLint 9.39.5, compileall	Linter de Python, JS/React y compilación.

8. Planificación y Organización

La planificación adoptó una estrategia de desarrollo y pruebas **Bottom-Up** (de abajo hacia arriba):

1. Verificación de la capa de persistencia y esquema SQLite.
2. Verificación de lógica de negocio y esquemas DTO aislados.
3. Integración de la API REST mediante TestClient y verificación HTTP.
4. Validación E2E del sistema completo uniendo Backend y Frontend.

Control de Versiones: Repositorio Git con convención de commits por etapa registrados sobre la rama main (desviación de proceso registrada y aceptada en hallazgo VER-08).

9. Desarrollo del Sistema

Se implementaron exitosamente las 15 funcionalidades planificadas en la Etapa 2. Entre las decisiones de diseño arquitectónico más relevantes destacan:

• **Borrado Lógico (Soft Delete):** Uso de la columna deleted_at para permitir la restauración inmediata ('Deshacer') sin destruir integridad relacional.

• **Cálculo Dinámico de Overdue:** La condición de vencimiento nunca se almacena fija en la BD; se calcula dinámicamente comparando la fecha límite con el día actual para tareas no terminadas.

• **Migraciones Aditivas:** Actualizaciones de esquema sin recrear tablas principales mediante ALTER TABLE ... ADD COLUMN y sintaxis defensivas.

10. Estrategia de Verificación y Validación

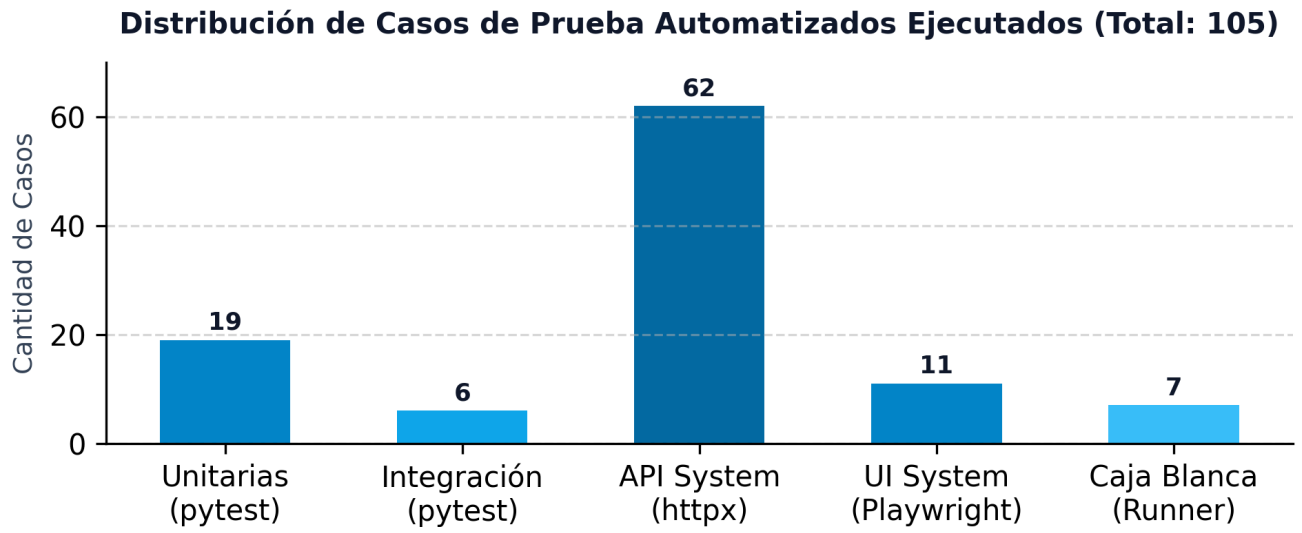
El proyecto aplicó la distinción conceptual entre Verificación (*¿construimos el producto correctamente?*) y Validación (*¿construimos el producto correcto?*):

A. Verificación Estática (Etapa 4): Inspección manual de código y ejecución de herramientas de análisis estático (Ruff, ESLint, compileall, Vite build). Se detectaron **10 hallazgos (VER-01 a VER-10)**, de los cuales 9 fueron corregidos (e.g. adición de restricciones CHECK en BD, manejo de tecla Escape) y 1 fue aceptado (uso de rama main).

B. Validación Dinámica (Etapas 5 a 10): Evaluación del sistema integrado ejecutando casos de prueba simulando escenarios de uso real.

11. Diseño y Estructura de Casos de Prueba

Se estructuró una suite de pruebas estandarizada documentada mediante fichas técnicas uniformes con campos: ID, Funcionalidad, Objetivo, Precondiciones, Datos de Entrada, Procedimiento, Resultado Esperado, Resultado Obtenido, Estado y Técnica.



12 y 13. Pruebas de Caja Negra y Caja Blanca

• **Caja Negra (73 casos de sistema):** Aplicación de Clases de Equivalencia y Análisis de Valores Límite sobre títulos (1, 100, 101 caracteres), descripciones (500, 501 chars), adjuntos (3 archivos, 5MB), prioridades y filtros.

• **Caja Blanca (7 casos):** Análisis de cobertura de instrucciones, decisiones y condiciones sobre unidades críticas: clean_title, move_task, delete_task/restore_task, create_attachment, update_subtask en backend; y validateTaskInput, dropIndex en frontend. Se alcanzó el **100% de cobertura de decisiones alcanzables**.

14. Pruebas Unitarias Automatizadas (Etapa 8)

Se desarrollaron **18 pruebas unitarias con pytest** (ampliadas a 19 tras TDD) sobre 16 funciones esenciales en schemas.py, repository.py y database.py.

Cada prueba unitaria se ejecuta aislada utilizando fixtures de bases SQLite temporales en memoria o /tmp sin requerir un servidor activo. **Resultado: 100% Aprobado (19/19 passed)**.

15. Pruebas de Integración (Etapa 9)

Se implementaron **6 pruebas de integración con pytest + TestClient de FastAPI**. Ejercitan el flujo completo: *Routers HTTP → Schemas → Repository → Database (SQLite) + Almacenamiento en Disco*.

ID	Integración Evaluada	Resultado Obtenido
CP-IN-01	Alta Enriquecida	Valida que el POST con subtareas, tags y links se persista y lea correctamente.
CP-IN-02	Archivos Adjuntos	Valida la subida multipart, escritura en disco física y descarga íntegra por HTTP.
CP-IN-03	Bloqueo de Finalización	Comprueba que intentar mover a 'Terminada' con subtareas devuelva HTTP 409.
CP-IN-04	Reapertura Automática	Comprueba que agregar una subtarea pendiente reabra una tarea terminada a 'En curso'.
CP-IN-05	Filtros y Ordenamiento	Valida filtros combinados por tag, overdue, prioridad y ordenamiento por fecha.
CP-IN-06	Historial de Actividad	Verifica la generación cronológica de registros en la tabla activity.

16. Pruebas de Sistema y Matriz de Trazabilidad

Se ejecutaron **80 casos de sistema** sobre la aplicación integrada (Backend Uvicorn en puerto 8000 + Frontend Vite en puerto 5173).

Matriz de Trazabilidad: Se verificó que los 23 Requisitos Funcionales (RF1 a RF23) cuentan con cobertura de al menos un caso de prueba aprobado. **Requisitos sin cobertura: 0.**

17. Consolidación de Automatización

El marco de automatización del proyecto consolidó un total de **104 casos automatizados** (105 tras TDD), distribuidos entre pytest, httpx y Playwright headless Chrome.

Ventajas constatadas: Ejecución reproducible de la suite completa en menos de 10 segundos, prevención instantánea de regresiones y confiabilidad durante refactorizaciones.

18. TDD y Refactorización (Etapa 12)

• **Desarrollo Dirigido por Pruebas (TDD):** Se desarrolló el helper puro is_overdue(due_date, status, today) siguiendo estrictamente el ciclo **RED** (creación de prueba fallida CP-UN-19), **GREEN** (implementación mínima) y **REFACTOR**.

• **Refactorización:** Se extrajo el método interno _log_set_changes en repository.py para eliminar duplicación de código en la gestión de etiquetas y enlaces, manteniendo intacta la API pública. Las 24 pruebas preexistentes permanecieron en verde.

19. Pruebas de Regresión (Etapa 13)

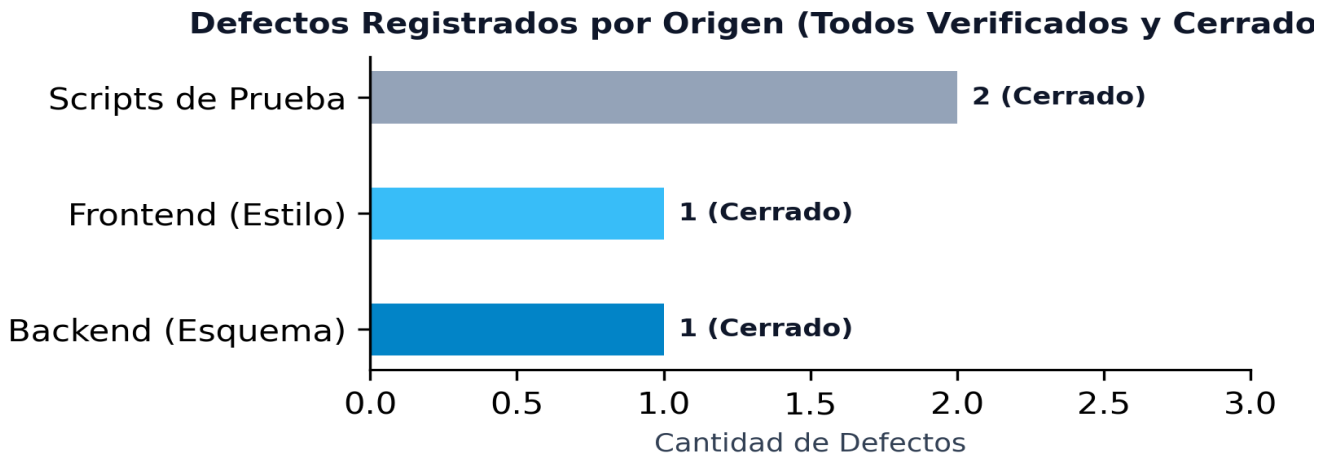
Tras la refactorización y TDD, se reejecutó la totalidad de la suite automatizada para garantizar que no se introdujeran regresiones en funcionalidades existentes.

Conjunto de Pruebas	Línea Base (Pre-TDD)	Resultado Regresión	Fallos	Veredicto
Suite Backend (pytest)	24 / 24 aprobados	25 / 25 aprobados	0 fallos	Sin Regresiones
Suite Sistema (httpx + Playwright)	80 / 80 aprobados	80 / 80 aprobados	0 fallos	Sin Regresiones
TOTAL CONSOLIDADOS	104 / 104 aprobados	105 / 105 aprobados	0 fallos	SIN REGRESIONES

20 y 21. Registro y Seguimiento de Defectos

Durante el proceso de verificación y pruebas se registraron **4 defectos formales (DEF-01 a DEF-04)**. Todos atravesaron el ciclo de vida completo (Nuevo → En Análisis → En Corrección → Resuelto → Verificado) y están totalmente cerrados:

ID	Defecto	Sev.	Prio.	Estado	Causa Raíz y Corrección Aplicada
DEF-01	Alta de Subtareas / Adjuntos	Alta	Alta	Verificado	Migración de esquema SQLite renombraba tabla rompiendo FKs. Corregido con migración aditiva.
DEF-02	Botón 'Nueva Tarea' sin estilo	Baja	Media	Verificado	Faltaba className='btn' en TaskForm.jsx. Se agregó la clase CSS.
DEF-03	Timeout en test CP-UI-09	Baja	Baja	Verificado	Locator compuesto de Playwright mal formado. Se usó el helper task_in.
DEF-04	Líneas largas en scripts	Baja	Baja	Verificado	Avisos de formato de Ruff en scripts. Se ajustó el formateo de líneas.



22. Revisión de Artefactos No Código (Etapa 14)

Mediante inspección cruzada (desk check) se revisaron los documentos de soporte del repositorio: README.md, COMANDOS_BACKEND.md y requirements-dev.txt.

Se detectaron y corrigieron **7 problemas (PRB-01 a PRB-07)** (2 de severidad alta, e.g. falta de declaración de dependencias de testing en requirements-dev.txt y falta de instrucciones para correr pytest).

23. Resultados Consolidados del Proyecto

Categoría de Prueba	Casos	Aprobados	Fallidos	Resultado Final
Pruebas de Caja Negra (API + UI)	73	73	0	100% Aprobado
Pruebas de Caja Blanca	7	7	0	100% Aprobado
Pruebas Unitarias (pytest)	19	19	0	100% Aprobado
Pruebas de Integración (pytest + TestClient)	6	6	0	100% Aprobado
Pruebas de Sistema Integrado	80	80	0	100% Aprobado
Suite de Regresión Final Consolidada	105	105	0	100% APROBADO

24. Conclusiones y Trabajos Futuros

¿El sistema desarrollado satisface las necesidades planteadas al inicio?

SÍ, rotundamente. Las evidencias cuantitativas respaldan esta conclusión: el 100% de los 23 Requisitos Funcionales (RF1 a RF23) están verificados y validados con 105 casos de prueba ejecutados y aprobados con 0

fallos y 0 defectos abiertos.

Lecciones Aprendidas de V&V;: La arquitectura en capas facilitó enormemente las pruebas aisladas; la verificación temprana en la Etapa 4 ahorró retrabajo mayor; y la automatización brindó la confianza necesaria para refactorizar.

Trabajos Futuros e Identificados: Incorporar autenticación de usuarios JWT, soporte de colaboración en tiempo real via WebSockets, accesibilidad por teclado alternativa al Drag & Drop, y cobertura de navegadores adicionales en Playwright.

25. Bibliografía y Anexos

- Consigna Oficial: ACTIVIDAD_INTEGRADORA_2026.pdf — Cátedra Verificación y Validación de Software.
- FastAPI Documentation: https://fastapi.tiangolo.com/
- React & Vite Documentation: https://react.dev/ | https://vite.dev/
- Pytest & Playwright Docs: https://docs.pytest.org/ | https://playwright.dev/
- Artefactos del proyecto: README.md, COMANDOS_BACKEND.md, carpetas docs/Etapa_1.md a Etapa_15.md, y archivos de evidencias en docs/evidencias/.